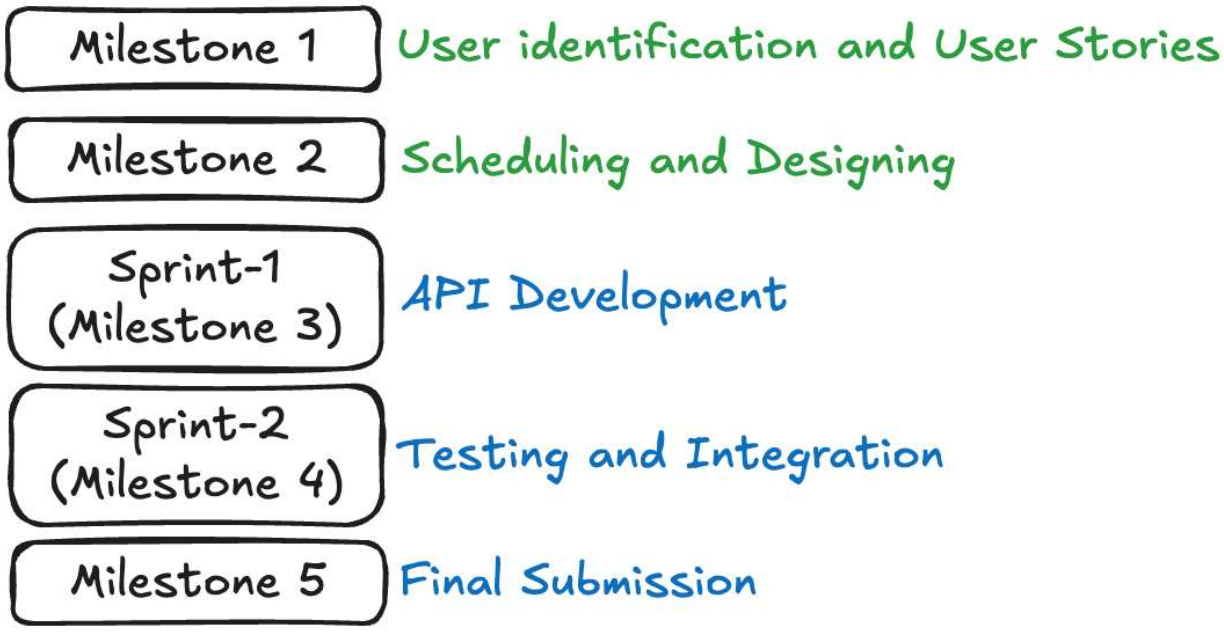
Software Engineering



Project Guide

End-to-End Workflow:



GENERAL INSTRUCTIONS:

Do's:

- **Communicate regularly** via standups, sprint reviews, and team tools (Slack, Jira, etc.).
- **Use Git effectively**: create branches for features, commit often, and write clear commit messages.
- **Maintain proper documentation** throughout the project.
- **Participate actively** in all phases, regardless of your role.
- **Review and test the code** before merging to the main branch.

Don'ts:

- Don't make decisions in isolation—always consult the team.
- Don't ignore documentation, it's critical for maintenance and grading.
- Don't delay syncing code or updates, merge frequently to avoid conflicts.

- Don’t rely on a single member for any task, collaborate and focus on teamwork.
- Any misconduct or abuse in the team will lead to permanent rejection.
- Any copying or plagiarism found strict action will be taken by the Disciplinary Committee.

Tips for Success

- **Regular Check-ins:** Hold short daily standups and weekly sprint reviews.
- **Version Control:** Use GitHub/GitLab consistently for all development and documentation.
- **Documentation First:** Keep a log of changes, design decisions, and learnings.
- **Peer Reviews:** Assign dedicated reviewers to ensure code quality and consistency.

Stage-1: Planning

Milestone 1: User Identification and User Stories

Focus: Identify application users and write SMART (Specific, Measurable, Achievable, Relevant, Time-boxed) user stories

Evaluation tip:
Your grade partly depends on how many user stories you commit to versus how many you deliver.

Team Responsibilities:

Role	Responsibilities
Project Manager	Lead the discussions and dive a consensus on feature requirements and user identification
Scrum Master	Document MoM and track discussion progress
Frontend Developer	Evaluate the feasibility of the frontend implementation
Backend Developer	Evaluate the feasibility of backend implementation

Everyone must stay actively engaged—later tasks build directly on the decisions you make here.

Conducting Interviews - Tips

- **There is no fixed number of interviews to conduct.**
  Keep researching until you uncover meaningful pain points or ideas that can inspire useful and realistic features.
- Focus on **user challenges, behaviors, and workarounds** rather than directly asking them what features they want.

Deliverables:

1. Identification of Users (Primary, secondary, and tertiary users.)

2. User Research Interviews and/or observation studies.
   Focus on identifying pain points (not suggesting features directly).
3. User Stories written using SMART guidelines.

Do & Don't Checklist

Do	Don't
Follow the **SMART** template for every story.	Over-promise: Remember, you have roughly **8 weeks** total.
Run a quick feasibility check before writing a story.	Add features just because they "sound cool" without verifying practicality.
Conduct **interviews or observation studies** to identify **pain points**.	Directly ask users "Would you like X feature?" — this biases their input.
Keep interview questions **open-ended and neutral.**	Ask yes/no or biased questions.
Submit proof of user research (short video/audio clips).	Submit only written summaries with no verification.

Milestone 2: Scheduling and Design

Focus: Build a realistic timeline and sprint plan, UI completion, and backend design

Team Responsibilities:

Role	Responsibilities
Scheduling Lead: Project Manager	Plan project timeline and scrum cadences
Scheduling Co-Lead: Scrum Master	Support in Scheduling and tracking
Design Lead: Backend Developer	Develop class diagrams and component layout
Frontend Developer	Complete most UI pages; ensure pages are linked

Code Reviewer: [Assign]	Review the structure and adherence to design practices before merging to the main branch in version control.

1. *It is the responsibility of the Project Manager and Scrum master to discuss with the whole team before finalizing the schedule.*
2. *Every frontend developer works on their branch, then makes a merge request for the code reviewer*
3. *In case of any issue noticed by a code reviewer or any other member, they must use the issue tracking feature in GitHub. (Issue Tracking will be submitted in milestone 6)*

Deliverables :

Milestone 2 PDF Report: Consolidated report including all the below components.

1. Design of Components
2. Class Diagram
3. Databased Design diagram
4. Sprint Schedule
5. Scrum Meetings Schedule and Minutes
6. Screenshot of Gantt Chart
7. Screenshot of Kanban Board
8. Screenshots of the Frontend pages have been developed.
9. Zip the frontend code with a README containing installation & run instructions.

Do & Don't Checklist

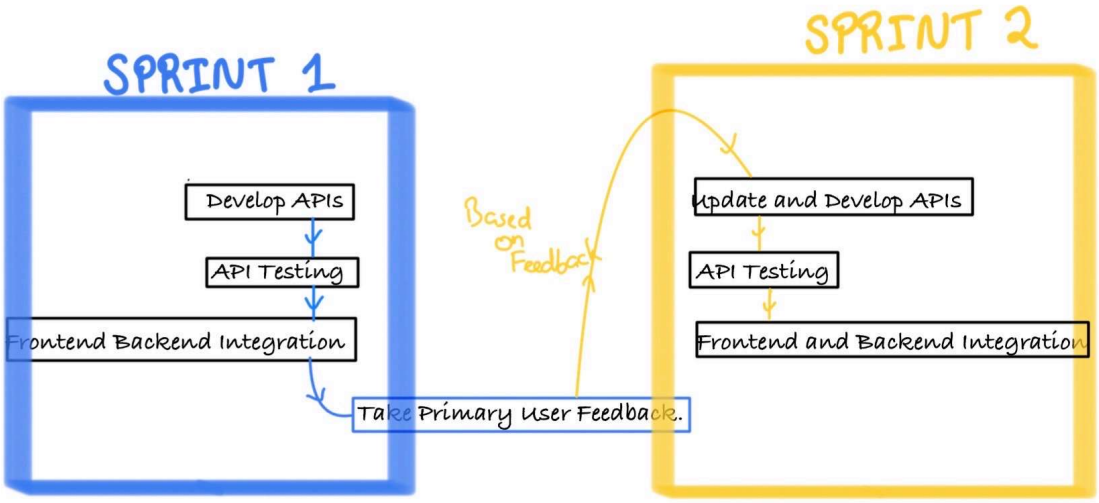
Do	Don't
Create **class diagrams** first, then code—update diagrams whenever design changes.	Skip diagrams and "figure it out in code," hoping to document later.
Work in feature branches; open a pull/merge request for every user story.	Push straight to `main` or share a branch among multiple devs.
Tag the assigned **Code Reviewer** in every merge request and address all comments before merging.	Merge if the reviewer is busy or has only "skimmed" the code.

Link each merge request to a **GitHub Issue** that describes the work (auto-close it on merge).	Let issues pile up without status updates or owners.

Stage-2: Implementation

Sprint 1-2 (Milestone 3-4): API Development, Testing, Integration, User Feedback

Focus: Design and development of REST API endpoints, Testing of APIs and Integrating backend and frontend, Taking primary user feedback, and implementing feedback in the next sprint.



Team Responsibilities:

Role	Responsibilities
Lead: Product Manager	Oversee and manage whole sprint from creating ,testing API's ,Integration and user feedback to implementation of feedback in next sprint
Backend Developer	Implement and document API endpoints
Tester	Unit Testing API's

Frontend Developer	API Integrations
Code Reviewer: [Assign]	Ensure API design consistency and code quality, Verify test coverage and integration completeness

Basic Unit testing must be done by the developer just after writing the code for the API

Sprint 1 Deliverables

1. Documentation of APIs in a Swagger-compatible YAML file.
2. Proper test cases to test an API.
3. For each test case, include the input, expected output, and actual output.
4. You can also showcase any API where the actual and expected outputs differ. (This demonstrates how testing helps improve your API.)
5. Submit pytest code for these APIs.
6. User feedback and plan to implement next sprint

Sprint 2 Deliverables

1. Changes made based on user feedback
2. **Updated** documentation of API's in a Swagger-compatible YAML file.
3. Backend code of the implemented API's
   Note: YAML file must have all the error handling, User stories mapping and description of each API listed.
4. Proper test cases to test an **Updated API**
5. Submit pytest code for the **Updated API**

Do & Don't Checklist

Do	**Don't**
Design each endpoint in **Swagger/YAML first**; confirm parameters & responses with the team.	"Prototype" in code without updating the YAML spec.
Follow **consistent URL naming** (`/api/v1/...`), HTTP verbs, status codes, and error format.	Return 200 for everything or mix snake_case and camelCase randomly.
Use **meaningful error messages** and document them in the spec.	Throw generic "Internal Server Error" without detail.

Write basic unit tests (pytest) immediately after coding an endpoint.	Plan to add tests after integration.
Record any discovered bugs as GitHub Issues to be showcased in Milestone 5.	Fix silently without a trace—loses learning evidence.
Create a **test matrix** listing endpoints, positive & negative cases, and integration flows.	Test ad-hoc in Postman and hope coverage is "good enough."
Automate repeatable tests with **pytest** + **requests** (or similar).	Rely solely on manual clicking through the UI.
Document **input, expected output, actual output** for each showcased API in the PDF.	Paste raw console logs with no explanation.
Highlight at least one case where **expected ≠ actual** and explain the fix.	Hide failures to make the report look "perfect."
Take **screenshots** (or terminal grabs) of pytest results for 4–5 key APIs.	Submit **.py** files or huge test dumps, the grader can't read quickly.

Milestone 5: Final Submission

Focus: Submit final working prototype with detailed documentation

Team Responsibilities:

Role	Responsibilities
Lead: Product Manager	Ensure all final documentation and assets are compiled for submission

Deliverables:

1. Video presentation (anyone from the team can record a demo of the working of the Project.
2. presentation (please include a brief overview of the whole project, mainly focusing on user stories and user identification)
3. Complete code of the working Project in zip format.
4. Readme file, which has all the instructions to run the project.
5. The final report, which has a complete compilation of all the milestones from 1 to 6.
    a. Including tools and technologies used
    b. Issue tracking that you have used during the project

Tools / Tech Stack

Category	Suggested Tools	Milestones
Planning & Docs	Google Docs / Sheets / Notion	M1, M2, M3, M6
	Trello / Jira	M1 - M6
UI/UX Design	Figma	M2, M3
	Draw.io / Lucidchart	M3
Version Control	Git + GitHub	M3 - M6
Frontend Dev	VueJS / ReactJS	M3 - M6
	Tailwind / MUI	M4
	Swagger	M4
Backend Dev	Flask / FastAPI	M3 - M6
Database	SQLite / PostgreSQL	M3 - M6
Testing	Postman	M4, M5
	Pytest	M5
Presentation	PowerPoint / Slides	M6
	OBS Studio	M6